

Compare high and low-altitude spectra and calc loss rate

Compare the ELFIN and THEMIS-A spectra during the 2022 Pet Rock Day Storm and calculate radiation belt loss rates. Let's make a summary plot first.

Regarding units, I wasted too much of myu time tracking them down. So I've chosen to use `cm` for distance units, and `keV` for energy units.

```
In [1]: import dateutil.parser
import pathlib
from datetime import datetime
import logging
import warnings

import matplotlib.pyplot as plt
import matplotlib.patches
import matplotlib.transforms as transforms
from mpl_toolkits.axes_grid1.axes_divider import make_axes_locatable
# import matplotlib.colors
import matplotlib.dates
import pyspedas
import pytpot
import scipy.stats
import scipy.optimize
import manylabls
import pandas as pd
import numpy as np
import asilib
import asilib.asi
import cartopy.crs as ccrs
import cartopy.feature as cfeature
import shapely.geometry
import IRBEM

import elfinasi

logger = logging.getLogger()
logger.disabled = True

# import pyparsing
# warnings.filterwarnings(
#     "ignore",
#     category=pyparsing.PyparsingDeprecationWarning,
#     append=False, # highest priority
# )
# warnings.filterwarnings(
#     "ignore",
#     category=DeprecationWarning,
#     module=r"pyparsing(\.|$)",
```

```
#     append=False,
# )
# warnings.filterwarnings(
#     "ignore",
#     message=r".*pyparsing.*deprecated.*",
#     append=False,
# )
```

```
In [2]: ELECTRON_CHARGE = 1.60217663E-19 # Coulombs
ELECTRON_MASS = 9.1093837E-31 # kg
PERMITTIVITY = 8.8541878188E-12 # C^2·kg^-1·m^-3·s^2
SPEED_OF_LIGHT = 3E10 # cm/s
ELECTRON_REST_ENERGY = 511 # keV
R_E = 6378.137 # km
```

Load and analyze THEMIS probe spectra

```
In [3]: time_range = ('2022-09-04T03:50', '2022-09-04T04:50')
_time_range = [dateutil.parser.parse(t_i) for t_i in time_range]
coordinates = 'gsm'
themis_probe = 'a'

themisa_spectra_time_ranges = (
    [datetime(2022, 9, 4, 4, 12, 0), datetime(2022, 9, 4, 4, 14, 0)],
    [datetime(2022, 9, 4, 4, 25, 0), datetime(2022, 9, 4, 4, 30, 0)],
)
themisd_spectra_time_ranges = (
    [datetime(2022, 9, 4, 4, 16, 0), datetime(2022, 9, 4, 4, 19, 0)],
    [datetime(2022, 9, 4, 4, 23, 0), datetime(2022, 9, 4, 4, 25, 0)],
)
themis_colors = plt.cm.viridis(np.linspace(0, 1, len(themisa_spectra_time_ranges)))
```

```
In [4]: themis_probes = np.array(['a', 'd', 'e'])
themis_Ew = ('fff_32_edc34', 'fff_32_edc34', 'fff_32_edc12')[np.where(themis_probes
themis_Bw = ('fff_32_scm3', 'fff_32_scm3', 'fff_32_scm3')[np.where(themis_probes==t
```

```
In [5]: no_update = False

fgm_vars = pyspedas.themis.fgm(probe=themis_probe, trange=time_range, time_clip=True
sst_vars = pyspedas.themis.sst(probe=themis_probe, trange=time_range, time_clip=True
mom_vars = pyspedas.themis.mom(probe=themis_probe, trange=time_range, time_clip=True
fft_vars = pyspedas.themis.fft(probe=themis_probe, trange=time_range, time_clip=True
state_vars = pyspedas.themis.state(probe=themis_probe, trange=time_range, time_clip

fgm_xr = pyplot.get_data(f'th{themis_probe}_fgl_{coordinates}')
fgm_times = np.array([t_i.replace(tzinfo=None) for t_i in pyspedas.time_datetime(fg
vi_xr = pyplot.get_data(f'th{themis_probe}_peim_velocity_{coordinates}'))
sst_xr = pyplot.get_data(f'th{themis_probe}_psef_en_eflux')
sst_xr_metadata = pyplot.get_data(f'th{themis_probe}_psef_en_eflux', metadata=True)
sst_flux_units = sst_xr_metadata['CDF']['VATT']['UNITS'].split()[0]
valid_sst_channels = np.where(~np.isnan(sst_xr.v[0, :]))[0]
ssta_times = np.array([t_i.replace(tzinfo=None) for t_i in pyspedas.time_datetime(s
ssta_spectrum = 1E3*sst_xr.y[:, valid_sst_channels]/sst_xr.v[0, valid_sst_channels]
```

```

bw_xr = pyplot.get_data(f'th{themis_probe}_{themis_Bw}')
ew_xr = pyplot.get_data(f'th{themis_probe}_{themis_Ew}')

print(sst_flux_units)

```

eV/(cm²-s-sr-eV)

```

In [6]: fig, ax = plt.subplots(5, sharex=True, figsize=(10, 7))

_lines = ax[0].plot(fgm_times, fgm_xr.y)
ax[0].legend(iter(_lines), [f'$B_{{{i}}}$' for i in ['x', 'y', 'z']], loc='upper le
ax[0].axhline(0, c='k', ls='--')
ax[0].set(
    ylim=(-25, 1.1*np.max(fgm_xr.y)),
    ylabel=f'$B_{{{coordinates.upper()}}}$ [nT]',
    xlim=_time_range
)

_lines = ax[1].plot(pyspedas.time_datetime(vi_xr.times), vi_xr.y)
ax[1].legend(iter(_lines), [f'$v_{{{i}}}$' for i in ['x', 'y', 'z']], loc='upper l
ax[1].axhline(0, c='k', ls='--')
ax[1].set(ylim=(-1.1*np.nanmax(vi_xr.y), 1.1*np.nanmax(vi_xr.y)), ylabel='ion veloc

for valid_e_channel in valid_sst_channels:
    ax[2].plot(
        ssta_times,
        ssta_spectrum[:, valid_e_channel],
        label=int(sst_xr.v[0, valid_e_channel]/1000)
    )
ax[2].set(yscale='log', ylabel=f'Electron flux\n[{sst_flux_units}]')
ax[2].legend(fontsize=8, ncols=3, title='Energy [keV]', loc='upper left', bbox_to_a

p = ax[3].pcolormesh(
    np.array(pyspedas.time_datetime(bw_xr.times)),
    bw_xr.v,
    bw_xr.y.T,
    norm=matplotlib.colors.LogNorm(vmax=1E-5),
    shading='nearest'
)
plt.colorbar(p, ax=ax[3])
ax[3].set(yscale='log', ylabel=f'th{themis_probe}_{themis_Bw}\n[Hz]')
f_ce = np.abs(ELECTRON_CHARGE)*1E-9*np.linalg.norm(fgm_xr.y, axis=1)/(2*np.pi*ELECT

p = ax[4].pcolormesh(
    pyspedas.time_datetime(ew_xr.times),
    ew_xr.v,
    ew_xr.y.T,
    norm=matplotlib.colors.LogNorm(vmax=1E-7)
)
plt.colorbar(p, ax=ax[4])
ax[4].set(yscale='log', ylabel=f'th{themis_probe}_{themis_Ew}\n[Hz]')

for ax_i in ax[3:]:
    ax_i.plot(pyspedas.time_datetime(fgm_xr.times), f_ce, label=f'$f_{{{ce}}}$', c='w
    ax_i.plot(pyspedas.time_datetime(fgm_xr.times), f_ce/2, label=f'$f_{{{ce}}}/2$',
    ax_i.plot(pyspedas.time_datetime(fgm_xr.times), f_ce/10, label=f'$f_{{{ce}}}/10$'

```

```

ax_i.set_facecolor('k')
ax[3].legend(loc='upper left')

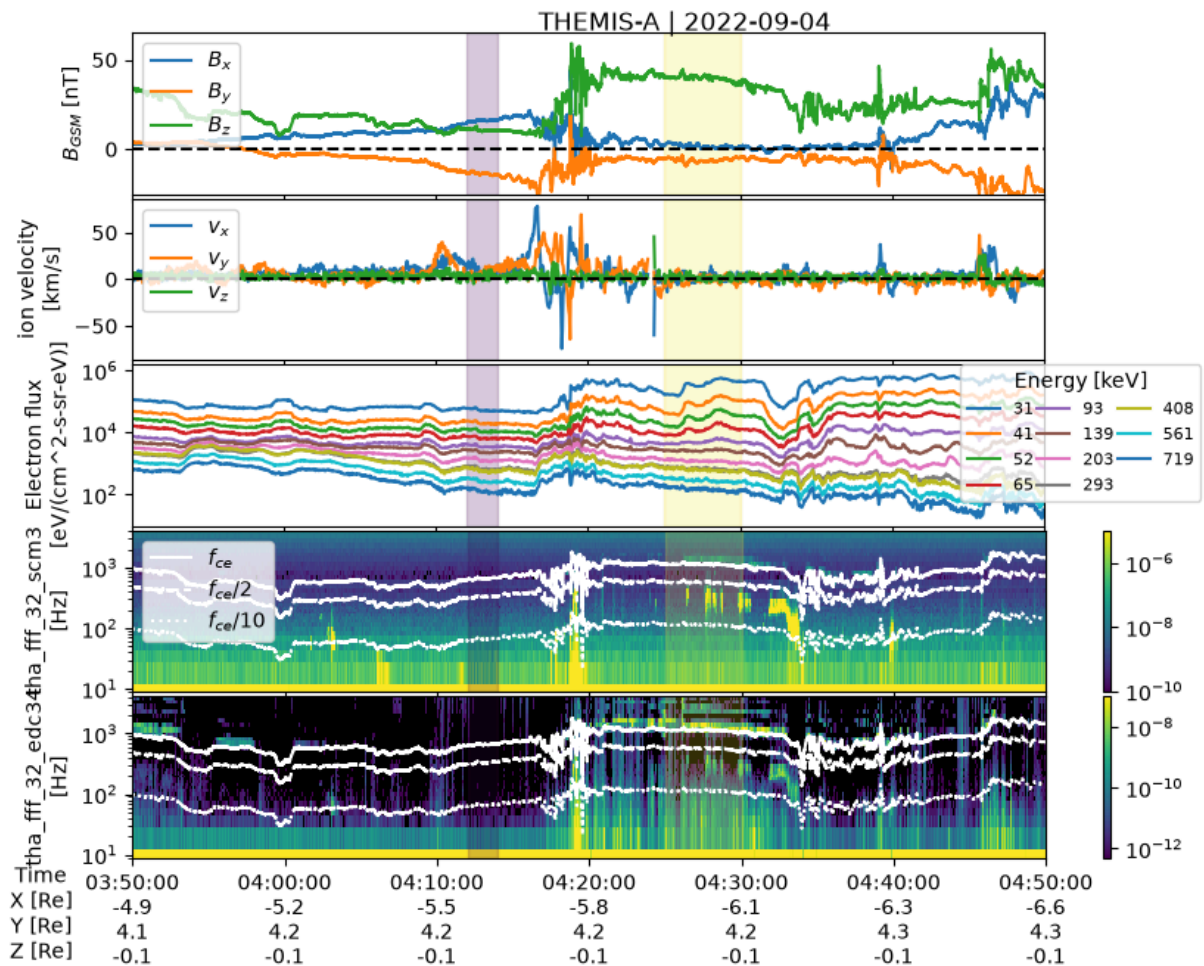
plt.suptitle(f'THEMIS-{themis_probe.upper()} | {time_range[0][:10]}')

state_xr = pyplot.get_data(f'th{themis_probe}_pos_{coordinates}')
state_times = pyspedas.time_datetime(state_xr.times)
state_times = [state_time.replace(tzinfo=None) for state_time in state_times]
state_df = pd.DataFrame(
    index=state_times,
    data={f'{component.upper()} [Re]':state_xr.y[:, i]/R_E for i, component in enum
)
manylabls.ManyLabels(ax[-1], state_df)

for ax_i in ax[:3]:
    divider = make_axes_locatable(ax_i)
    cax = divider.append_axes("right", size="24%", pad=0.08)
    cax.remove()
plt.subplots_adjust(hspace=0.03, top=0.959)

for themis_spectra_time_range, color in zip(themisa_spectra_time_ranges, themis_col
for ax_i in ax:
    ax_i.axvspan(*themis_spectra_time_range, color=color, alpha=0.2)

```



What fraction of the total field was Bz before and after the dipolarization?

```
In [7]: for i, themis_time_range in enumerate(themisa_spectra_time_ranges):
        idx = np.where((np.array(fgm_times)>=themis_time_range[0]) & (np.array(fgm_time
        mean_b = fgm_xr.y[idx, :].mean(axis=0)
        mean_bxy = fgm_xr.y[idx, 1:].mean(axis=0)

        print(
            f'{themis_time_range=}\n|B| = {np.linalg.norm(mean_b) :.2f} nT, '
            f'|Bz|/|B| = {mean_b[-1]/np.linalg.norm(mean_b) :.2f}, '
            f'|Bz|/|Bxy| = {mean_b[-1]/np.linalg.norm(mean_bxy) :.2f}\n'
        )
```

```
themis_time_range=[datetime.datetime(2022, 9, 4, 4, 12), datetime.datetime(2022, 9,
4, 4, 14)]
|B| = 23.59 nT, |Bz|/|B| = 0.46, |Bz|/|Bxy| = 0.62
```

```
themis_time_range=[datetime.datetime(2022, 9, 4, 4, 25), datetime.datetime(2022, 9,
4, 4, 30)]
|B| = 40.28 nT, |Bz|/|B| = 0.99, |Bz|/|Bxy| = 0.99
```

Calc THEMIS Spectra

```
In [8]: fgm_times = np.array([t_i.replace(tzinfo=None) for t_i in pyspedas.time_datetime(fg
bidx_before = np.where(
    (fgm_times>=themisa_spectra_time_ranges[0][0]) &
    (fgm_times<=themisa_spectra_time_ranges[0][1])
)[0]
bidx_after = np.where(
    (fgm_times>=themisa_spectra_time_ranges[-1][0]) &
    (fgm_times<=themisa_spectra_time_ranges[-1][1])
)[0]
mag_b = np.linalg.norm(fgm_xr.y, axis=1)
mean_b_before = mag_b[bidx_before].mean()
mean_b_after = mag_b[bidx_after].mean()
print(f'B field before: {mean_b_before:.2f} nT and after: {mean_b_after:.2f} nT.')
```

B field before: 23.60 nT and after: 40.31 nT.

```
In [9]: themis_spectra_mean = np.full((len(themis_spectra_time_ranges), len(sst_xr.v[0], va
themis_spectra_iqr = np.full((len(themis_spectra_time_ranges), 2, len(sst_xr.v[0],
for i, themis_spectra_time_range in enumerate(themis_spectra_time_ranges):
    idx = np.where(
        (ssta_times>=themis_spectra_time_range[0]) &
        (ssta_times<=themis_spectra_time_range[1])
    )[0]
    themis_spectra_mean[i, :] = np.mean(ssta_spectrum[idx, :], axis=0)
    themis_spectra_iqr[i, ...] = np.quantile(ssta_spectrum[idx, :], (0.25, 0.75), a
```

```
In [10]: fig, ax = plt.subplots()
        for i, (themis_time_range, color) in enumerate(zip(themis_spectra_time_ranges, the
        ax.errorbar(sst_xr.v[0, valid_sst_channels], themis_spectra_mean[i, :], color=
            markersize=7, elinewidth=1, marker='.', ls='-')
        ax.text(
            0.01, 0.2-0.03*i, f'{themis_time_range[0]:%H:%M:%S} - {themis_time_range[1]
```

```

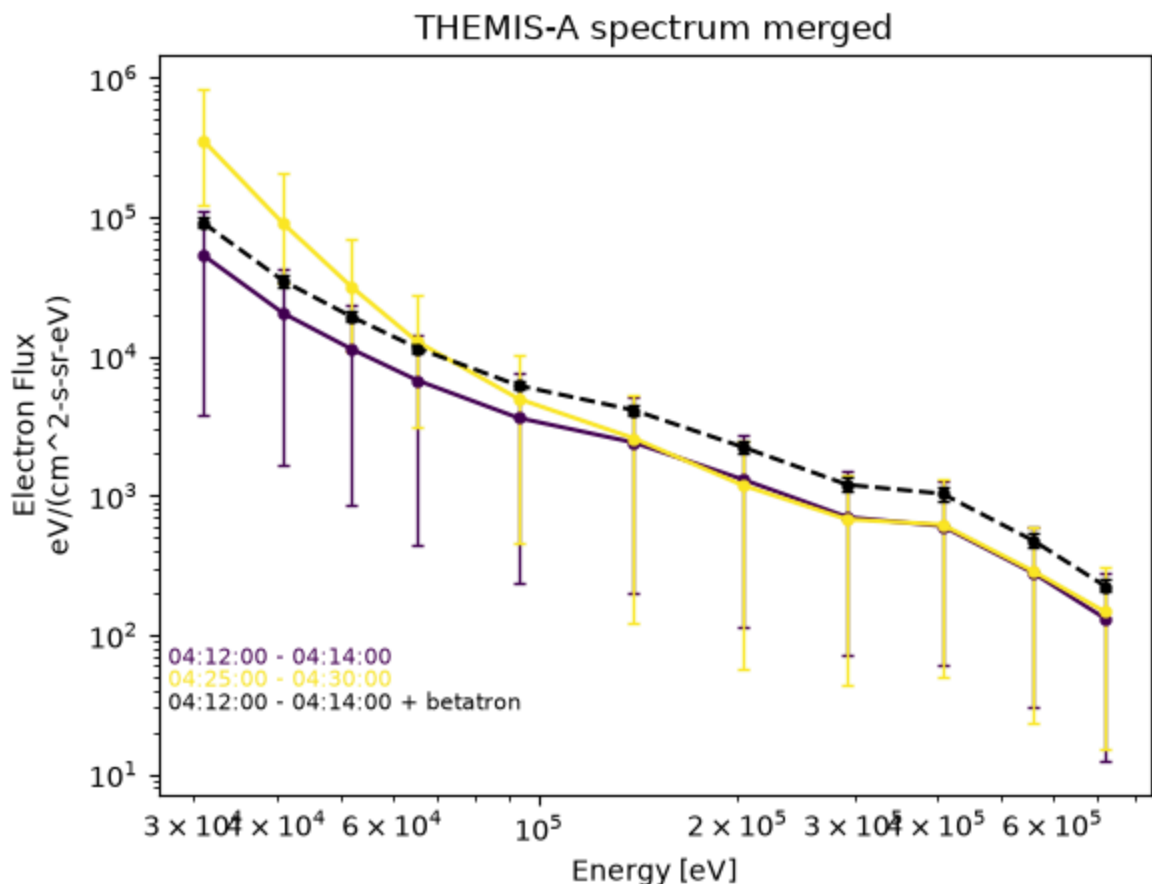
        fontsize=8, transform=ax.transAxes, va='top', ha='left', color=color
    )

ax.errorbar(
    sst_xr.v[0, valid_sst_channels],
    (mean_b_after/mean_b_before)*themis_spectra_mean[0, :],
    color='k',
    yerr=(mean_b_after/mean_b_before)*np.abs(themis_spectra_mean[0, :]-themis_spectra_mean[0, :]),
    capsize=2,
    markersize=7,
    elinewidth=1,
    marker='.',
    ls='--'
)

ax.text(
    0.01, 0.2-0.03*(i+1), f'{themis_spectra_time_ranges[0][0]:%H:%M:%S} - {themis_spectra_time_ranges[0][1]:%H:%M:%S}',
    fontsize=8, transform=ax.transAxes, va='top', ha='left', color='k'
)

ax.set(xlabel='Energy [eV]', yscale='log', xscale='log', title='THEMIS-A spectrum merged')
ax.set_ylabel(f'Electron Flux\n{sst_flux_units}');

```



So it looks like betatron acceleration does not explain the increase in the fluxes. This seems like an injection!

Load and analyze ELFIN spectra

```
In [11]: elfin_spectra_time_ranges = (
    [datetime(2022, 9, 4, 4, 18, 3), datetime(2022, 9, 4, 4, 20, 5)],
    [datetime(2022, 9, 4, 4, 20, 5), datetime(2022, 9, 4, 4, 20, 25)],
    [datetime(2022, 9, 4, 4, 20, 25), datetime(2022, 9, 4, 4, 21, 20)],
)
elfin_colors = plt.cm.jet(np.linspace(0, 1, len(elfin_spectra_time_ranges)))
```

```
In [12]: elfin_probe = 'a'
pad_obj = elfinasi.EPD_PAD(
    elfin_probe, time_range, start_pa=0, min_counts=None, accumulate=1, spin_time_t
    lc_exclusion_angle=0
)
transformed_state = pad_obj.transform_state()
```

07-Aug-26 14:07:09: /home/msshumko/research/.venv/lib/python3.13/site-packages/cdfli
b/epochs.py:180: DeprecationWarning: The 'generic' unit for NumPy timedelta is depre
cated, and will raise an error in the future. This includes implicit conversion of b
are integers (e.g. `+ 1`).Please use a specific unit instead.

```
total_datetime = np.array(sum(arrays))
```

07-Aug-26 14:07:09: /home/msshumko/research/.venv/lib/python3.13/site-packages/cdfli
b/epochs.py:181: DeprecationWarning: The 'generic' unit for NumPy timedelta is depre
cated, and will raise an error in the future. This includes implicit conversion of b
are integers (e.g. `+ 1`).Please use a specific unit instead.

```
total_datetime = np.where(nat_positions, np.datetime64("NaT"), total_datetime)
```

07-Aug-26 14:07:09: /home/msshumko/research/.venv/lib/python3.13/site-packages/cdfli
b/epochs.py:181: DeprecationWarning: The 'generic' unit for NumPy timedelta is depre
cated, and will raise an error in the future. This includes implicit conversion of b
are integers (e.g. `+ 1`).Please use a specific unit instead.

```
total_datetime = np.where(nat_positions, np.datetime64("NaT"), total_datetime)
```

07-Aug-26 14:07:37: /home/msshumko/research/.venv/lib/python3.13/site-packages/cdfli
b/epochs.py:180: DeprecationWarning: The 'generic' unit for NumPy timedelta is depre
cated, and will raise an error in the future. This includes implicit conversion of b
are integers (e.g. `+ 1`).Please use a specific unit instead.

```
total_datetime = np.array(sum(arrays))
```

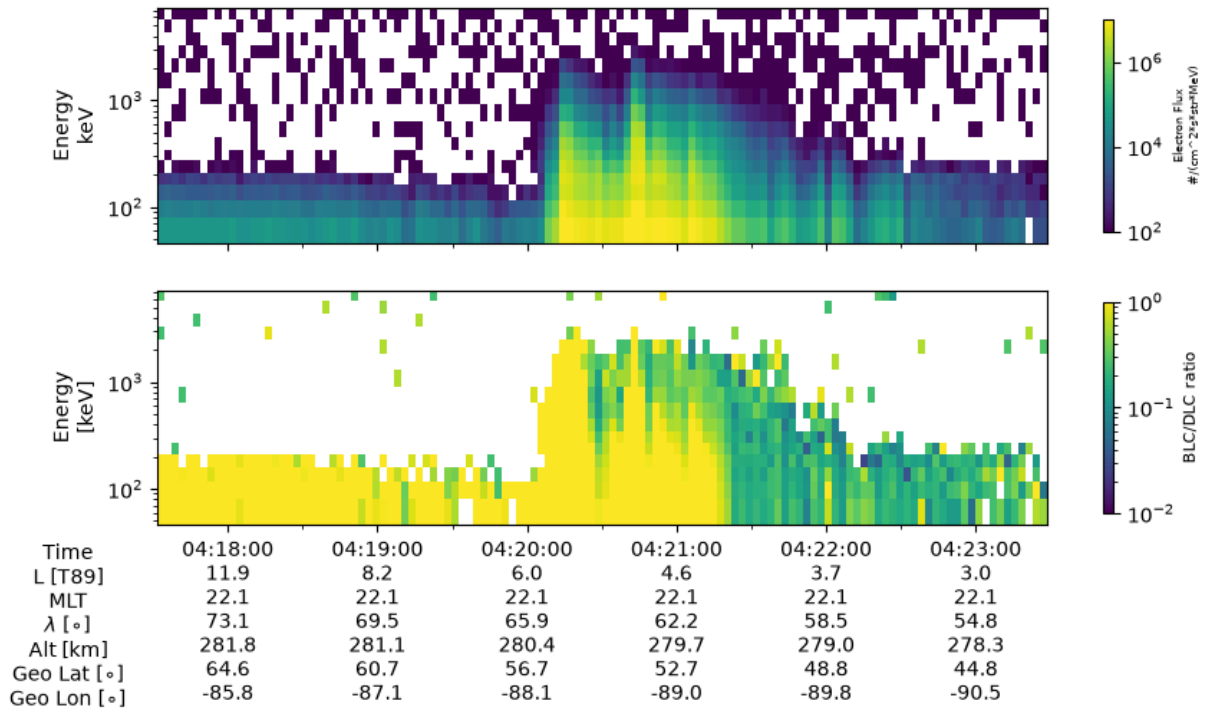
07-Aug-26 14:07:37: /home/msshumko/research/.venv/lib/python3.13/site-packages/cdfli
b/epochs.py:181: DeprecationWarning: The 'generic' unit for NumPy timedelta is depre
cated, and will raise an error in the future. This includes implicit conversion of b
are integers (e.g. `+ 1`).Please use a specific unit instead.

```
total_datetime = np.where(nat_positions, np.datetime64("NaT"), total_datetime)
```

```
In [43]: fig, cx = plt.subplots(2, 1, sharex=True, figsize=(10, 6))

pad_obj.plot_omni(cx[0], labels=True, colorbar=True, vmin=1E2, vmax=1E7, pretty_plo
# pad_obj.plot_pad_scatter(cx[1])
pad_obj.plot_blc_dlc_ratio(cx[1], labels=True, colorbar=True, cmap='viridis', vmin=
pad_obj.plot_position(cx[1])
cx[1].xaxis.set_major_locator(matplotlib.dates.SecondLocator(bysecond=0))
cx[1].xaxis.set_minor_locator(matplotlib.dates.SecondLocator(bysecond=30))
```

```
# cx[1].xaxis.set_label_coords(-0.05, -0.007*7)
# cx[1].xaxis.label.set_size(10)
# for elfin_spectra_time_range, color in zip(elfin_spectra_time_ranges, elfin_color)
#     mixed_transform = transforms.blended_transform_factory(cx[0].transData, cx[0].transTime)
#     cx[0].plot(
#         [elfin_spectra_time_range[0], elfin_spectra_time_range[1]], [0.1,0.1], color
#     )
```

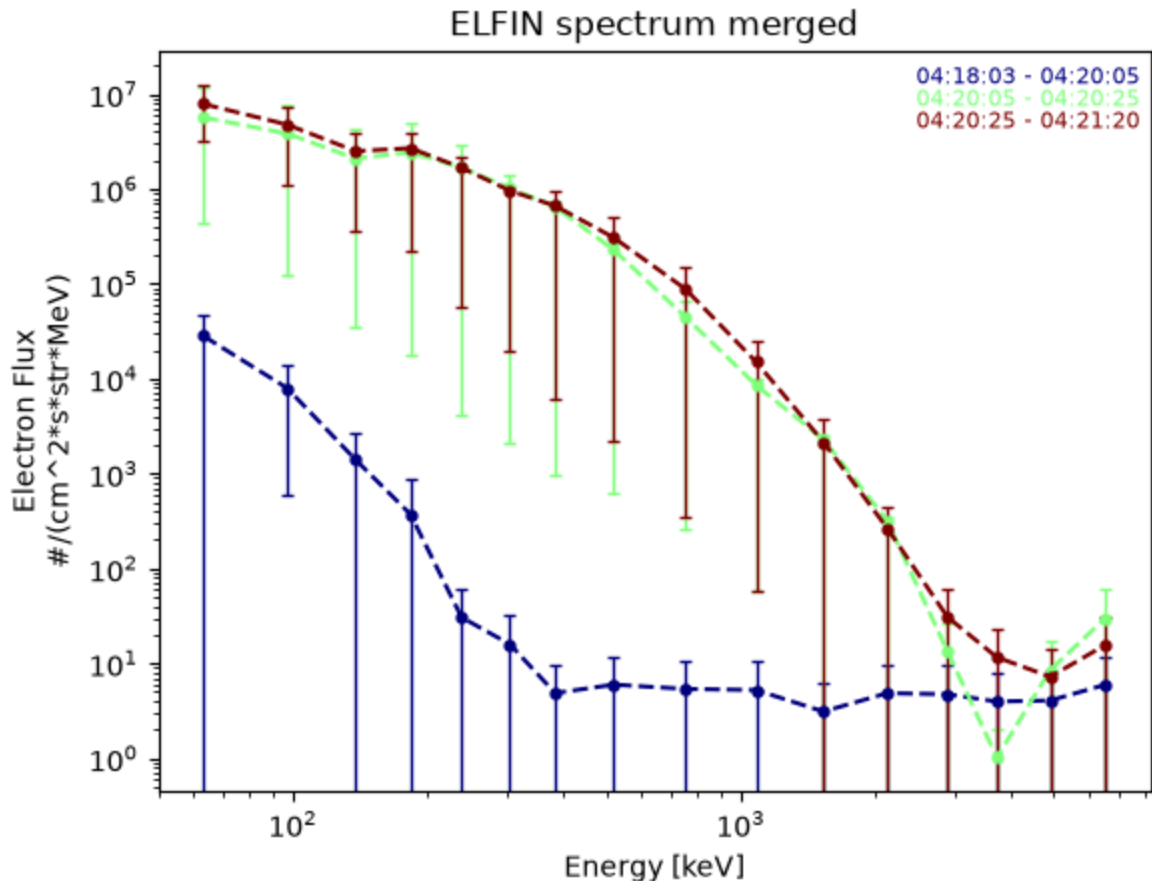


```
In [14]: elfin_spectra_mean = np.nan*np.zeros((len(elfin_spectra_time_ranges), pad_obj.energy)
elfin_spectra_iqr = np.nan*np.zeros((len(elfin_spectra_time_ranges), 2, pad_obj.energy)

for i, (elfin_spectra_time_range, color) in enumerate(zip(elfin_spectra_time_ranges, elfin_color)):
    pad_flt = pad_obj.pad.sel(time=slice(elfin_spectra_time_range[0], elfin_spectra_time_range[1]))
    elfin_spectra_mean[i, :] = pad_flt.mean(dim=('time', 'pa'), skipna=True).sel(energy=elfin_spectra_mean[i, :])
    elfin_spectra_iqr[i, :, :] = pad_flt.quantile([0.25, 0.75], dim=('time', 'pa'), skipna=True)
```

```
In [15]: fig, ax = plt.subplots()
for i, (elfin_spectra_time_range, color) in enumerate(zip(elfin_spectra_time_ranges, elfin_color)):
    ax.errorbar(pad_obj.energy, elfin_spectra_mean[i, :], color=color, yerr=np.abs(elfin_spectra_iqr[i, :, :]),
                markersize=7, elinewidth=1, marker='.', ls='--')
    ax.text(
        0.99, 0.98-0.03*i, f'{elfin_spectra_time_range[0]:%H:%M:%S} - {elfin_spectra_time_range[1]:%H:%M:%S}',
        fontsize=8, transform=ax.transAxes, va='top', ha='right', color=color
    )

ax.set(xlabel='Energy [keV]', yscale='log', xscale='log', title='ELFIN spectrum merged')
ax.set_ylabel(f'Electron Flux\n{pad_obj._flux_units}');
```



Both the IB and chorus-driven spectrum are essentially the same. Now let's make a similar plot, but this time fix the x- and y-axis range, and change units slightly.

This is to address the reviewer feedback.

See how the ELFIN spectra changes throughout the pass

First set up the ELFIN spectrum time range array and specify the energy range, in KeV, to fit across.

```
In [16]: elfin_spectra_time_range = (datetime(2022, 9, 4, 4, 19, 0), datetime(2022, 9, 4, 4, 4, 4))
elfin_spectral_times = pd.date_range(*elfin_spectra_time_range, freq='30s')

integrate_energies_kev = np.linspace(63, 719.5)
print(f'Integrating over {integrate_energies_kev[0]} - {integrate_energies_kev[-1]}')
```

Integrating over 63.0 - 719.5 keV.

```
In [17]: elfin_fit_idx = np.where((pad_obj.energy >= integrate_energies_kev[0]) & (pad_obj.energy <= integrate_energies_kev[-1]))
themis_fit_idx = np.where((sst_xr.v[0, :] >= 1e3 * integrate_energies_kev[0]) & (sst_xr.v[0, :] <= 1e3 * integrate_energies_kev[-1]))
```

```
In [18]: def fit_exponent(x_data, y_data):
        """
```

```

Fits an exponent function of the form  $y = j_0 \cdot \exp(-E/E_0)$ 

Parameters
-----
x_data: np.array
    The independent variable
y_data: np.array
    The dependent variable

Returns
-----
E0, J0: float
    The J0 and E0 parameters
"""
with np.errstate(divide='ignore'):
    valid_logy = np.isfinite(np.log(y_data))
if np.sum(valid_logy) < 2:
    warnings.warn(f'{np.sum(valid_logy)} (<2) valid log(y) points to fit to an
    return np.nan, np.nan
slope, intercept, _, _, _ = scipy.stats.linregress(x_data[valid_logy], np.log(y
return -1/slope, np.exp(intercept)

def fit_powerlaw(x_data, y_data):
    """
    Fits a power law function of the form  $y = j_0 \cdot E^{-\text{gamma}}$ 

    Parameters
    -----
    x_data: np.array
        The independent variable
    y_data: np.array
        The dependent variable

    Returns
    -----
    gamma, j0: float
        The spectral slope (gamma) and j0 parameters
    """
    with np.errstate(divide='ignore'):
        valid_logy = np.isfinite(np.log(y_data))
    if np.sum(valid_logy) < 2:
        warnings.warn(f'{np.sum(valid_logy)} (<2) valid log(y) points to fit to a p
        return np.nan, np.nan
    gamma, intercept, _, _, _ = scipy.stats.linregress(np.log(x_data)[valid_logy],
    return -gamma, np.exp(intercept)

def fit_elfin_spectrum(pad_obj, time_range, energy_range=(50, 200), spectral_type='
time_idx = np.where((pad_obj.pad.time >= time_range[0]) & (pad_obj.pad.time <=
energy_channel_idx = np.where((pad_obj.energy >= energy_range[0]) & (pad_obj.en

if pa == 'blc-ablc':
    _flux_timeseries = (pad_obj.blc-pad_obj.ablc)[time_idx][:, energy_channel_i
    _spectrum = np.mean(_flux_timeseries, axis=0)
elif pa == 'omni':
    omni_fluxes = pad_obj.pad.mean(dim='pa')
    _spectrum = np.mean(omni_fluxes[time_idx, :][:, energy_channel_idx], axis=0

```

```

else:
    raise NotImplementedError(
        f'Unknown pa: {pa}. Can only fit a "blc-ablc" or "omni" spectrum.'
    )
_spectrum /= 1e3 # Convert from #/(cm^2 sr s MeV) to #/(cm^2 sr s keV)
if spectral_type == 'exp':
    return fit_exponent(pad_obj.energy[energy_channel_idx], _spectrum)
elif spectral_type == 'powerlaw':
    return fit_powerlaw(pad_obj.energy[energy_channel_idx], _spectrum)
else:
    raise NotImplementedError(
        f'Unknown spectral_type: {spectral_type}. Can only fit an "exponential"
    )

def fit_themis_spectrum(_sst, time_range, energy_range=(50, 200), spectral_type='ex
sst_times = np.array([t_i.replace(tzinfo=None) for t_i in pyspedas.time_datetim
time_idx = np.where((sst_times >= time_range[0]) & (sst_times <= time_range[1])
energy_idx = np.where(~np.isnan(_sst.v[0, :]) & (_sst.v[0, :]/1e3 >= energy_ran
# Convert THEMIS SST from energy flux units: eV/(cm^2 sr s eV) to number flux u
sst_flux = 1E3*_sst.y[:, energy_idx]/_sst.v[0, energy_idx]

sst_energy = _sst.v[0, energy_idx]/1e3 # Convert to keV
sst_spin_avg_flux = sst_flux[time_idx, :]
mean_spectrum = np.mean(sst_spin_avg_flux, axis=0)

if spectral_type == 'exp':
    return fit_exponent(sst_energy, mean_spectrum)
elif spectral_type == 'powerlaw':
    return fit_powerlaw(sst_energy, mean_spectrum)
else:
    raise NotImplementedError(
        f'Unknown spectral_type: {spectral_type}. Can only fit an "exponential"
    )

```

```

In [19]: elfin_fit_params = np.full((len(elfin_spectral_times)-1, 4), np.nan, dtype=object)
for i, (start_time, end_time) in enumerate(zip(elfin_spectral_times[:-1], elfin_spe
elfin_fit_params[i, 0:2] = (start_time, end_time)
elfin_fit_params[i, 2:] = fit_elfin_spectrum(
    pad_obj,
    (start_time, end_time),
    energy_range=(integrate_energies_kev[0], integrate_energies_kev[-1]),
    spectral_type='powerlaw'
)

elfin_fit_params = pd.DataFrame(elfin_fit_params, columns=['start_time', 'end_time'

themis_fit_params = np.full((len(themisa_spectra_time_ranges), 4), np.nan, dtype=ob
for i, (start_time, end_time) in enumerate(themisa_spectra_time_ranges):
    themis_fit_params[i, 0:2] = (start_time, end_time)
    themis_fit_params[i, 2:] = fit_themis_spectrum(
        sst_xr,
        (start_time, end_time),
        energy_range=(integrate_energies_kev[0], integrate_energies_kev[-1]),
        spectral_type='powerlaw'
    )

```

```
)
themis_fit_params = pd.DataFrame(themis_fit_params, columns=['start_time', 'end_time', 'gamma', 'j0'])
```

```
07-Aug-26 14:07:37: /tmp/ipykernel_73670/4099185440.py:42: RuntimeWarning: invalid value encountered in log
  valid_logy = np.isfinite(np.log(y_data))
```

```
In [20]: elfin_fit_params.style.format({'J0 [# /cm^2-s-sr-keV]': '{:.2e}'})
```

```
Out[20]:
```

	start_time	end_time	gamma	J0 [# /cm^2-s-sr-keV]
0	2022-09-04 04:19:00	2022-09-04 04:19:30	4.783292	1.62e+10
1	2022-09-04 04:19:30	2022-09-04 04:20:00	4.409161	1.71e+09
2	2022-09-04 04:20:00	2022-09-04 04:20:30	1.371855	2.40e+06
3	2022-09-04 04:20:30	2022-09-04 04:21:00	1.120676	1.30e+06
4	2022-09-04 04:21:00	2022-09-04 04:21:30	2.180331	7.25e+07
5	2022-09-04 04:21:30	2022-09-04 04:22:00	3.013293	4.33e+07
6	2022-09-04 04:22:00	2022-09-04 04:22:30	4.345610	1.20e+10
7	2022-09-04 04:22:30	2022-09-04 04:23:00	3.625394	5.33e+07

```
In [21]: themis_fit_params.style.format({'J0 [# /cm^2-s-sr-keV]': '{:.2e}'})
```

```
Out[21]:
```

	start_time	end_time	gamma	J0 [# /cm^2-s-sr-keV]
0	2022-09-04 04:12:00	2022-09-04 04:14:00	1.527628	4.19e+06
1	2022-09-04 04:25:00	2022-09-04 04:30:00	1.715231	1.33e+07

```
In [22]: fig, fx = plt.subplots(1, 1, figsize=(7, 5))

cmap = plt.get_cmap('viridis')
colors = cmap(np.linspace(0, 1, len(elfin_spectral_times)-1))
themis_colors = ['k', 'orange']
elfin_spectra_mean = np.nan*np.zeros((len(elfin_spectral_times)-1, pad_obj.energy.s
elfin_spectra_iqr = np.nan*np.zeros((len(elfin_spectral_times)-1, 2, pad_obj.energy

for i, (start_time, end_time, color) in enumerate(zip(elfin_spectral_times[:-1], el
    pad_flt = pad_obj.pad.sel(time=slice(start_time, end_time))
    elfin_spectra_mean[i, :] = pad_flt.mean(dim=('time', 'pa'), skipna=True).sel(en
    # yerr = np.abs(elfin_spectra_mean[i, :] - elfin_spectra_iqr[i, :]),
    # elfin_spectra_iqr[i, ...] = pad_flt.quantile((0.25, 0.75), dim=('time', 'pa'))
    fx.errorbar(pad_obj.energy, elfin_spectra_mean[i, :], color=color, yerr=None, c
        markersize=4, elinewidth=1, marker='D', lw=1,
        label=f'ELA {start_time:%H:%M:%S}-{end_time:%H:%M:%S}')

gamma, j0 = elfin_fit_params.loc[i, ['gamma', 'J0 [# /cm^2-s-sr-keV]']]
fx.plot(pad_obj.energy[elfin_fit_idx], j0*pad_obj.energy[elfin_fit_idx]**(-gamm

# e0, j0 = fit_elfin_spectrum(pad_obj, (start_time, end_time), energy_range=fit
```

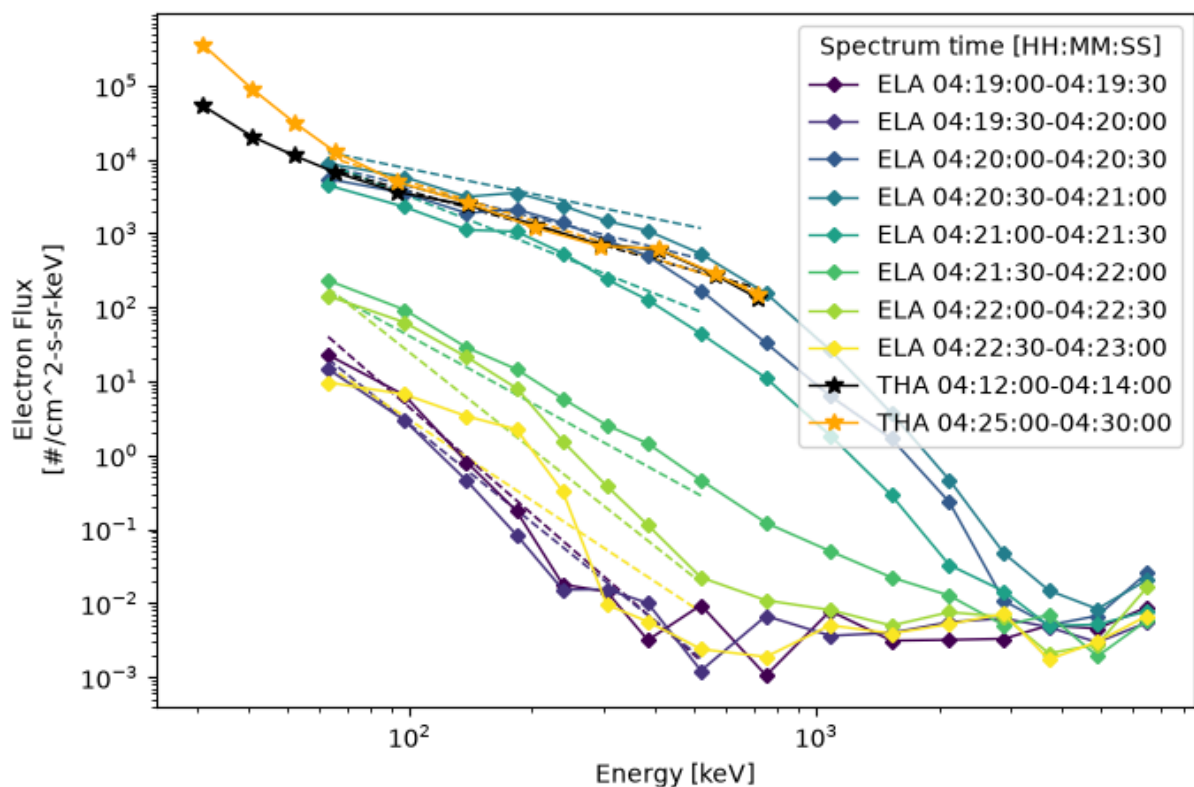
```

# fx.plot(pad_obj.energy[elfin_fit_idx], j0*np.exp(-pad_obj.energy[elfin_fit_idx]
for i, (themis_time_range, color) in enumerate(zip(themis_spectra_time_ranges, the
fx.errorbar(sst_xr.v[0, valid_sst_channels]/1E3, themis_spectra_mean[i, :], co
            markersize=7, elinewidth=1, marker='*', lw=1,
            label=f'THA {themis_time_range[0]:%H:%M:%S}-{themis_time_range[1]:%H:%M
gamma, j0 = themis_fit_params.loc[i, ['gamma', 'j0 [# /cm^2-s-sr-keV]']]
fx.plot(integrate_energies_kev, j0*integrate_energies_kev**(-gamma), color=colo

fx.set(xlabel='Energy [keV]', yscale='log', xscale='log')
fx.legend(loc='upper right', title='Spectrum time [HH:MM:SS]', fontsize=10, title_f
fx.set_ylabel(f'Electron Flux\n[# /cm^2-s-sr-keV]')
plt.suptitle(f'ELFIN mean electron spectrum');
plt.tight_layout()

```

ELFIN mean electron spectrum



Looks like the THEMIS-A and ELFIN-A spectra agree really well between 04:20:15-04:21:15 which corresponds to the outer RB and the IB. So this means that the trapped fluxes were isotropic, and there was not a strong L-shell dependence on the fluxes.

RB loss estimate

Let's now estimate how many electrons were lost to both hemispheres during this time. Thanks to Google Gemini, here is how we'll calculate it.

$$N_{trapped} = \int_V \int_E \int_\Omega \underbrace{\left(\frac{j(E, \Omega)}{v(E)} \right)}_{\left[\frac{\text{partic./sec}}{\text{cm}^2 \text{sr s keV}} \right]} \underbrace{d\Omega}_{[\text{sr}]} \underbrace{dE}_{[\text{keV}]} \underbrace{dV}_{[\text{cm}^3]} \longrightarrow T_{\text{obs}} \cdot A_{\text{foot}} \cdot P_{\text{prec}} \cdot \Omega_{\text{loss}}$$

where

$$v(E) = c \sqrt{1 - \left(\frac{E_0}{E + E_0} \right)^2}$$

is the relativistic velocity, and we integrate the number of trapped particles contained in the flux tube volume

$$V_{\text{sector}} = (B_{\text{iono}} \cdot A_{\text{iono}}) \int_{s_2}^{s_1} \frac{1}{B(s)} ds$$

You can intuitively think this from the conservation of magnetic flux (and because $\rho_e \sim B$). The A_{iono} is the ionospheric flux tube footprint area which we reference to B_{iono} . Since the the flux tube footprint area decreases with B (gyroradius gets smaller), it conversely gets larger with smaller B . Therefore, as we trace the field line to the magnetic equator and then to the conjugate footprint, the small area contributions scale with $1/B$.

Now the rate of particles lost is

$$\frac{N_{\text{lost}}}{dt} = 2 \cdot \pi \cdot A_{\text{iono}} \cdot \int_{E_{\text{min}}}^{E_{\text{max}}} j_{\text{ELFIN}}(E) dE$$

where $j_{\text{ELFIN}}(E)$ is the ELFIN flux with units $\mu\text{A}/(\text{cm}^2 \text{sr s keV})$, factor of 2 accounts for losses in both hemispheres, and $\pi [\text{sr}]$ assumes isotropic precipitation within the loss cone.

Putting this all together, the fraction lost every second is

$$f = \frac{\frac{N_{\text{lost}}}{dt}}{N_{\text{trapped}}}$$

and the number of seconds to completely empty the loss cone is

$$\tau = \frac{N_{\text{trapped}}}{\frac{N_{\text{lost}}}{dt}}$$

Let's first calculate the precipitation area and the flux tube volume

```
In [23]: themis_location_codes = ['GILL', 'ATHA', 'PINA', 'TPAS', 'KAPU']
asis = asilib.Imagers([
    asilib.asi.themis(loc, time='2022-09-04T04:21:24', alt=110) for loc in themis_l
])
```

```
In [24]: area_vertices = [
    (-115, 55), (-82, 57), (-78, 51), (-100, 51)
]
central_lonlat = np.mean(area_vertices, axis=0)
poly_lonlat = shapely.geometry.Polygon(area_vertices)
src_crs = ccrs.PlateCarree()
tgt_crs = ccrs.AlbersEqualArea(
    central_longitude=central_lonlat[0],
    central_latitude=central_lonlat[1]
)

# 3. Transform the geometry coordinates to the target projection
poly_projected = tgt_crs.project_geometry(poly_lonlat, src_crs)

# 4. Extract the area (returned in square meters)
area_sq_meters = poly_projected.area
area_sq_cm = area_sq_meters * 1e4
area_sq_km = area_sq_meters / 1e6
print(f'Lower-bound area of the expansion phase aurora is {area_sq_km:.2e} [km^2]')
```

Lower-bound area of the expansion phase aurora is area_sq_km=1.15e+06 [km^2]

Now let's visualize the precipitation area

```
In [25]: center = (
    np.mean(asis.lon_bounds), np.mean(asis.lat_bounds)
)
projection = ccrs.Orthographic(
    central_longitude=center[0],
    central_latitude=center[1]
)
fig = plt.figure(figsize=(5, 5))
ax = fig.add_subplot(projection=projection)
ax.add_feature(cfeature.LAND, color='grey')
ax.add_feature(cfeature.OCEAN, color='cyan')
ax.add_feature(cfeature.COASTLINE, edgecolor='k')
ax.gridlines(linestyle=':')
ax.set_global()
ax.set_extent(
    (center[0]-20, center[0]+20, center[1]-10, center[1]+8),
    crs=ccrs.PlateCarree()
)

gl = ax.gridlines(
    crs=ccrs.PlateCarree(), draw_labels=True,
    linewidth=2, color='gray', alpha=0.5, linestyle='--'
)

asis.plot_map(ax=ax, color_bounds=asis.imagers[0].auto_color_bounds())

shape = matplotlib.patches.Polygon(
    area_vertices, closed=True, facecolor=None, edgecolor='purple', lw=3, ls='--'
)
ax.add_patch(shape)
```

```
base, exp = f"{area_sq_km:.2e}".split("e")
ax.text(
    central_lonlat[0],
    central_lonlat[1],
    f'$A_{{iono}} = {base} \\times 10^{{int(exp)}} \\ [\\mathrm{{km^2}}]$',
    color='white',
    fontsize=10,
    fontweight='bold',
    transform=ccrs.Geodetic(),
    ha='center',
    va='center'
);
```

```
07-Aug-26 14:07:39: /home/msshumko/research/.venv/lib/python3.13/site-packages/cartopy/mpl/ticker.py:151: MatplotlibDeprecationWarning: The locs attribute was deprecated in Matplotlib 3.11 and will be removed in 3.13.  
    self.locs, degs, mins, secs = self._get_dms(self.locs)  
  
07-Aug-26 14:07:39: /home/msshumko/research/.venv/lib/python3.13/site-packages/cartopy/mpl/ticker.py:151: MatplotlibDeprecationWarning: The locs attribute was deprecated in Matplotlib 3.11 and will be removed in 3.13.  
    self.locs, degs, mins, secs = self._get_dms(self.locs)  
  
07-Aug-26 14:07:41: /home/msshumko/research/.venv/lib/python3.13/site-packages/cartopy/mpl/ticker.py:151: MatplotlibDeprecationWarning: The locs attribute was deprecated in Matplotlib 3.11 and will be removed in 3.13.  
    self.locs, degs, mins, secs = self._get_dms(self.locs)  
  
07-Aug-26 14:07:41: /home/msshumko/research/.venv/lib/python3.13/site-packages/cartopy/mpl/ticker.py:151: MatplotlibDeprecationWarning: The locs attribute was deprecated in Matplotlib 3.11 and will be removed in 3.13.  
    self.locs, degs, mins, secs = self._get_dms(self.locs)  
  
07-Aug-26 14:07:41: /home/msshumko/research/.venv/lib/python3.13/site-packages/cartopy/mpl/ticker.py:151: MatplotlibDeprecationWarning: The locs attribute was deprecated in Matplotlib 3.11 and will be removed in 3.13.  
    self.locs, degs, mins, secs = self._get_dms(self.locs)  
  
07-Aug-26 14:07:41: /home/msshumko/research/.venv/lib/python3.13/site-packages/cartopy/mpl/ticker.py:151: MatplotlibDeprecationWarning: The locs attribute was deprecated in Matplotlib 3.11 and will be removed in 3.13.  
    self.locs, degs, mins, secs = self._get_dms(self.locs)  
  
07-Aug-26 14:07:41: /home/msshumko/research/.venv/lib/python3.13/site-packages/cartopy/mpl/ticker.py:151: MatplotlibDeprecationWarning: The locs attribute was deprecated in Matplotlib 3.11 and will be removed in 3.13.  
    self.locs, degs, mins, secs = self._get_dms(self.locs)  
  
07-Aug-26 14:07:41: /home/msshumko/research/.venv/lib/python3.13/site-packages/cartopy/mpl/ticker.py:151: MatplotlibDeprecationWarning: The locs attribute was deprecated in Matplotlib 3.11 and will be removed in 3.13.  
    self.locs, degs, mins, secs = self._get_dms(self.locs)  
  
07-Aug-26 14:07:41: /home/msshumko/research/.venv/lib/python3.13/site-packages/cartopy/mpl/ticker.py:151: MatplotlibDeprecationWarning: The locs attribute was deprecated in Matplotlib 3.11 and will be removed in 3.13.  
    self.locs, degs, mins, secs = self._get_dms(self.locs)  
  
07-Aug-26 14:07:41: /home/msshumko/research/.venv/lib/python3.13/site-packages/cartopy/mpl/ticker.py:151: MatplotlibDeprecationWarning: The locs attribute was deprecated in Matplotlib 3.11 and will be removed in 3.13.  
    self.locs, degs, mins, secs = self._get_dms(self.locs)
```

```
py/mpl/ticker.py:151: MatplotlibDeprecationWarning: The locs attribute was deprecated in Matplotlib 3.11 and will be removed in 3.13.
```

```
self.locs, degs, mins, secs = self._get_dms(self.locs)
```

```
07-Aug-26 14:07:41: /home/msshumko/research/.venv/lib/python3.13/site-packages/cartopy/mpl/ticker.py:151: MatplotlibDeprecationWarning: The locs attribute was deprecated in Matplotlib 3.11 and will be removed in 3.13.
```

```
self.locs, degs, mins, secs = self._get_dms(self.locs)
```

```
07-Aug-26 14:07:41: /home/msshumko/research/.venv/lib/python3.13/site-packages/cartopy/mpl/ticker.py:151: MatplotlibDeprecationWarning: The locs attribute was deprecated in Matplotlib 3.11 and will be removed in 3.13.
```

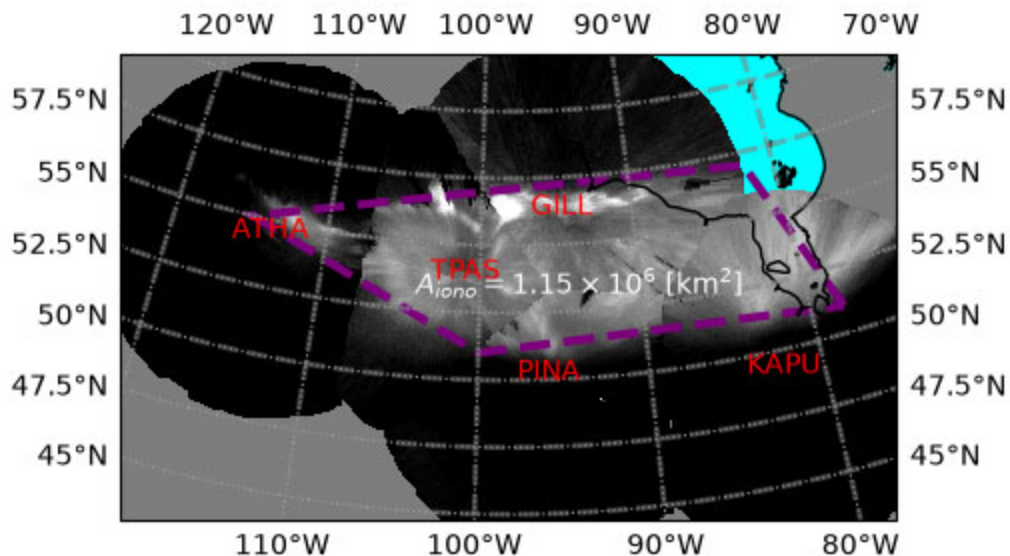
```
self.locs, degs, mins, secs = self._get_dms(self.locs)
```

```
07-Aug-26 14:07:41: /home/msshumko/research/.venv/lib/python3.13/site-packages/cartopy/mpl/ticker.py:151: MatplotlibDeprecationWarning: The locs attribute was deprecated in Matplotlib 3.11 and will be removed in 3.13.
```

```
self.locs, degs, mins, secs = self._get_dms(self.locs)
```

```
07-Aug-26 14:07:41: /home/msshumko/research/.venv/lib/python3.13/site-packages/cartopy/mpl/ticker.py:151: MatplotlibDeprecationWarning: The locs attribute was deprecated in Matplotlib 3.11 and will be removed in 3.13.
```

```
self.locs, degs, mins, secs = self._get_dms(self.locs)
```



```
In [26]: def flux_tube_volume(irbem_model, X, maginput, starting_area, R0=1, interpNum=100_000)
        """
        Calculate the volume of a flux tube volume of a magnetic field line, scaled to
        starting area (typically taken in the ionosphere).

        Parameters
        -----
        irbem_model: IRBEM.MagFields
            An instance of the IRBEM.MagFields class, which is used to compute magnetic
        X: dict
            A dictionary that specifies the input time and location. The `time` key can
            ISO-formatted time string, or a `datetime.datetime` or `pd.TimeStamp` object.
            The three location keys: `x1`, `x2`, and `x3` specify the location in the `
        maginput: dict
            The magnetic field input dictionary. See the online documentation for the v
            keys and the corresponding models.
        starting_area: float
            The cross-sectional area of the flux tube at the starting point, with cm^2
        R0: float
            The radius, in units of RE, of the reference surface (i.e. altitude) between
            the line is traced.
        interpNum: int
            The number of samples to interpolate the magnetic field line.
            100000 is a good balance between speed and accuracy.

        Returns
        -----
        float
            Flux tube volume(s) in cubic cm.
        """
        field_line_dict = irbem_model.interpolate_field_line(X, maginput, R0=R0, alpha

        # If the mirror point is below the ground, Scipy will error, try
        # to change the R0 parameter...
        try:
            startInd = scipy.optimize.brentq(field_line_dict['fB'], 0,
                                              len(field_line_dict['S'])/2)
            endInd = scipy.optimize.brentq(field_line_dict['fB'],
                                              len(field_line_dict['S'])/2, len(field_line_dict

        except ValueError as err:
            if str(err) == 'f(a) and f(b) must have different signs':
                raise ValueError('Mirror point below R0') from err
            else:
                raise

        # Resample S to a finer density of points.
        if len(field_line_dict['S']) > interpNum:
            warnings.warn('Warning: interpolating with less data than IRBEM outputs, '+
                          'the bounce period may be inaccurate!')
        sInterp = np.linspace(startInd, endInd, num = interpNum)

        # Calculate the small change in position, and magnetic field.
        dx = np.convolve(field_line_dict['fx'](sInterp), [-1,1], mode = 'same')
        dy = np.convolve(field_line_dict['fy'](sInterp), [-1,1], mode = 'same')
        dz = np.convolve(field_line_dict['fz'](sInterp), [-1,1], mode = 'same')
```

```

ds = R_E*1e5*np.sqrt(dx**2 + dy**2 + dz**2)
dB = field_line_dict['fB'](sInterp) + field_line_dict['mirrorB']
starting_flux = starting_area*field_line_dict['mirrorB']
V = starting_flux*np.trapezoid(1/dB, dx=ds[1:])
return V

```

In [27]: `mag_model = IRBEM.MagFields(kext='T89')`

In [28]: `tube_V = flux_tube_volume(
 mag_model,
 {'time':elfin_spectral_times[0], 'x1':100, 'x2':central_lonlat[1], 'x3':central
 {'Kp':50},
 starting_area=area_sq_cm
)
 print(f'Flux tube volume at {elfin_spectral_times[0]} is {tube_V:.2e} cm^3.')`

Flux tube volume at 2022-09-04 04:19:00 is 3.05e+28 cm³.

Sanity check. How many cubic Re fit into the flux tube?

In [29]: `R_E_cm = R_E*1e5
 int(tube_V/R_E_cm**3)`

Out[29]: 117

Okay! 117 Cubic Earth's appears appropriate. How many trapped electrons were there?

In [30]: `_gamma_themis, _j0_themis = themis_fit_params.loc[1, ['gamma', 'J0 [# /cm^2-s-sr-keV
 v = SPEED_OF_LIGHT*np.sqrt(
 1-(ELECTRON_REST_ENERGY/(ELECTRON_REST_ENERGY+integrate_energies_kev))**2
) # cm/s
 # 1e6 is to convert from eV to MeV, 4pi is to account for the solid angle of the tr
 N_trapped = 4*np.pi*tube_V*np.trapezoid(
 y=_j0_themis*integrate_energies_kev**(-_gamma_themis)/v,
 x=integrate_energies_kev
)
 print(
 f'Number of trapped electrons between {integrate_energies_kev[0]} keV and {inte
)`

Number of trapped electrons between 63.0 keV and 719.5 keV is 1.7e+25 electrons.

1.7×10^{25} trapped electrons seem to be appropriate to previous studies

1. [Millan+2002](#),
2. [Blum+2013](#)
3. [O'Brien+2004](#)
4. [Lorentzen+2001](#)

Need to find more recent studies.

Now let us calculatate the loss rate

```
In [37]: loss_cone_sr = 2*np.pi*(1-np.cos(np.deg2rad(60))) # sr
```

```
In [31]: loss_spectrum_time = pd.to_datetime("2022-09-04T04:20:00")

# 2. Compute absolute time delta and locate the minimum
closest_row_index = (elfin_fit_params["start_time"] - loss_spectrum_time).abs().idx_
_j0_elfin, _gamma_elfin = elfin_fit_params.loc[closest_row_index, ['J0 [# /cm^2-s-sr

# np.pi corresponds to isotropic precipitation into the loss cone, and 2 correspond
dN_lost_dt = 2*np.pi*area_sq_cm*np.trapezoid(
    y=_j0_elfin*(integrate_energies_kev)**(-_gamma_elfin),
    x=integrate_energies_kev
)
print(
    f'Number of electrons lost per second in {integrate_energies_kev[0]}-{integrate_
    f'is: {dN_lost_dt:.1e} e-/s. The number of time to lose the trapped population
    )
```

Number of electrons lost per second in 63.0-719.5 keV is: 5.9e+22 e-/s. The number of time to lose the trapped population is: 4.7 minutes.

~5 minutes seems very fast! On the order of how long the expansion phase lasts. I am a little skeptical of this value.

Now let's compare the ELFIN - THEMIS-D spectrum

```
In [32]: fgmd_vars = pyspedas.themis.fgm(probe='d', trange=time_range, time_clip=True, no_up
sstd_vars = pyspedas.themis.sst(probe='d', trange=time_range, time_clip=True, no_up
assert (fgmd_vars is not None) and (sstd_vars is not None)
```

```
In [33]: fgmd_xr = pyplot.get_data(f'thd_fgl_{coordinates}')
fgmd_times = np.array([t_i.replace(tzinfo=None) for t_i in pyspedas.time_datetime(f
sstd_xr = pyplot.get_data(f'thd_psef_en_eflux')
valid_sst_channels = np.where(~np.isnan(sstd_xr.v[0, :]))[0]
sstd_times = np.array([t_i.replace(tzinfo=None) for t_i in pyspedas.time_datetime(s
sstd_spectrum = 1E6*sstd_xr.y[:, valid_sst_channels]/sstd_xr.v[0, valid_sst_channel
```

```
In [34]: spec_fig = plt.figure(figsize=(15, 8))
gs = spec_fig.add_gridspec(4, len(themisd_spectra_time_ranges)+2, hspace=1.8, right
themis_gs = gs[:, :2].subgridspec(2, 1, wspace=0.05, hspace=0.05)
elfin_gs = gs[2:4, :2].subgridspec(2, 1, wspace=0.01, hspace=0.05)
spec_grid = gs[:, 2:].subgridspec(len(elfin_spectra_time_ranges), len(themisd_spect

ax = [None]*4
ax[0] = spec_fig.add_subplot(themis_gs[0,:])
ax[1] = spec_fig.add_subplot(themis_gs[1,:], sharex=ax[0])
ax[2] = spec_fig.add_subplot(elfin_gs[0,:])
ax[3] = spec_fig.add_subplot(elfin_gs[1,:], sharex=ax[2])

bx = np.full((len(elfin_spectra_time_ranges), len(themisd_spectra_time_ranges)), np
for row in range(len(elfin_spectra_time_ranges)):
    for col in range(len(themisd_spectra_time_ranges)):
        if (row == 0) and (col == 0):
```

```

        bx[row, col] = spec_fig.add_subplot(spec_grid[row, col])
    else:
        bx[row, col] = spec_fig.add_subplot(spec_grid[row, col], sharex=bx[0, 0])

    if row != len(themisa_spectra_time_ranges):
        bx[row, col].tick_params('x', labelbottom=False)
    if col > 0:
        bx[row, col].tick_params('y', labelleft=False)

# PLOT THEMIS
_lines = ax[0].plot(pyspedas.time_datetime(fgmd_xr.times), fgmd_xr.y)
ax[0].legend(iter(_lines), [f'$B_{{{i}}}$' for i in ['x', 'y', 'z']], loc='upper left')
ax[0].axhline(0, c='k', ls='--')
ax[0].set(
    ylim=(-1.1*np.max(fgmd_xr.y), 1.1*np.max(fgmd_xr.y)),
    ylabel=f'$B_{{{coordinates.upper()}}}$ [nT]',
    xlim=time_range
)
ax[0].tick_params('x', labelbottom=False)

for valid_e_channel in valid_sst_channels:
    ax[1].plot(
        sstd_times,
        sstd_spectrum[:, valid_e_channel],
        label=int(sstd_xr.v[0, valid_e_channel]/1000)
    )
ax[1].set(yscale='log', ylabel='Electron flux\n#[/(cm^2 sr s MeV)]')
_legend = ax[1].legend(fontsize=8, ncols=3, title='Energy [keV]', loc='upper left',
for _obj in _legend.legend_handles:
    _obj.set_linewidth(2)
manylabels.ManyLabels(ax[1], state_df)
ax[1].xaxis.set_major_locator(plt.MaxNLocator(5))
ax[1].xaxis.set_label_coords(-0.1, -0.008*7)

for themis_spectra_time_range, color in zip(themisd_spectra_time_ranges, themis_colors):
    for ax_i in ax[2:]:
        ax_i.axvspan(*themis_spectra_time_range, color=color, alpha=0.5)

# PLOT ELFIN
pad_obj.plot_omni(ax[2], labels=True, vmin=1E2, vmax=1E7, pretty_plot=False, colorbar=True)
pad_obj.plot_blc_dlc_ratio(ax[3], labels=True, cmap='viridis', vmin=1E-2, vmax=1, colorbar=True)
pad_obj.plot_position(ax[3])
ax[2].tick_params('x', labelbottom=False)
ax[3].xaxis.set_major_locator(plt.MaxNLocator(5))
ax[3].xaxis.set_label_coords(-0.1, -0.007*7)
ax[3].xaxis.label.set_size(10)

for elfin_spectra_time_range, color in zip(elfin_spectra_time_ranges, elfin_colors):
    for ax_i in ax[2:4]:
        ax_i.axvspan(*elfin_spectra_time_range, color=color, alpha=0.5)

# Plot the spectra matrix
for i, (bx_row, elfin_color) in enumerate(zip(bx, elfin_colors)):
    for j, (bx_col, themis_color, themis_spectra_time_range) in enumerate(zip(bx_row, themis_spectra_time_ranges, themis_colors)):
        idx = np.where(

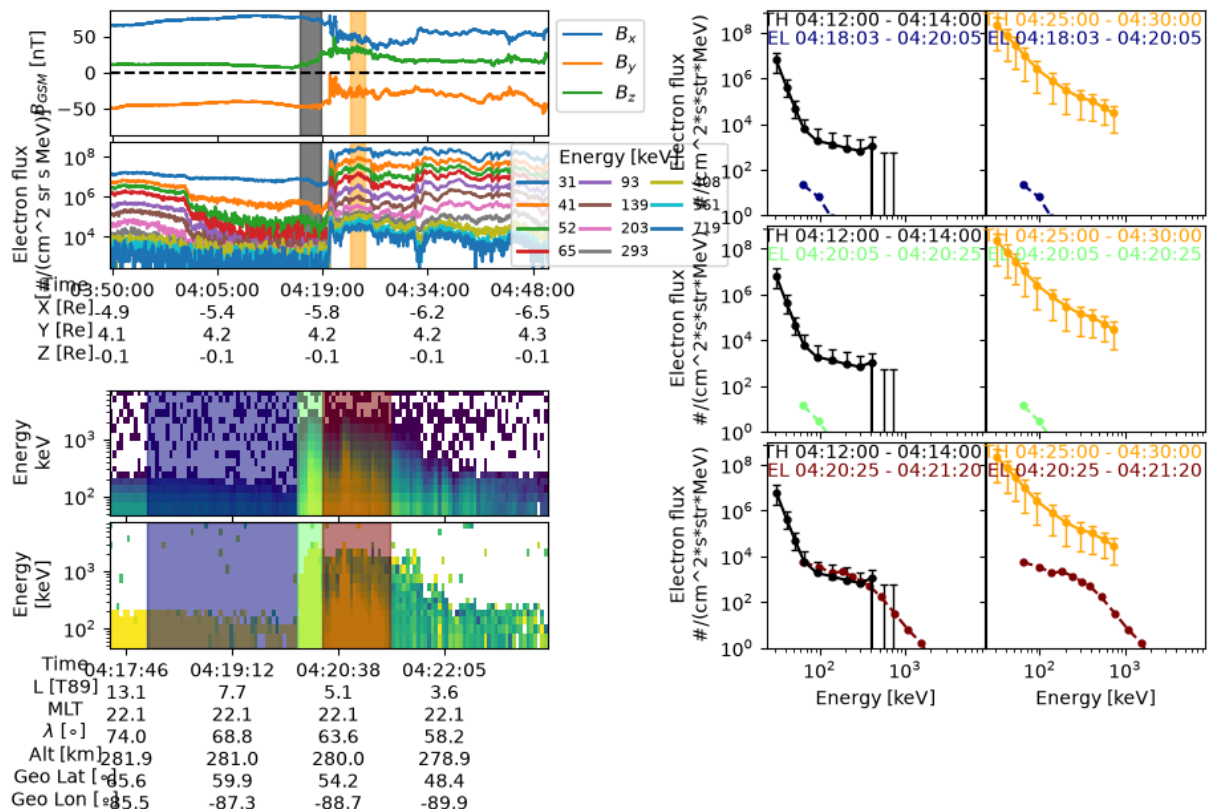
```

```

(sstd_times>=themis_spectra_time_range[0]) &
(sstd_times<=themis_spectra_time_range[1])
)[0]
themisd_spectra_median = np.median(sstd_spectrum[idx, :], axis=0)
themisd_spectra_iqr = np.nanquantile(sstd_spectrum[idx, :], (0.25, 0.75), a

bx_col.errorbar(pad_obj.energy, elfin_spectra_mean[i, :], color=elfin_color,
                markersize=7, elinewidth=1, marker='.', ls='--')
bx_col.errorbar(sstd_xr.v[0, valid_sst_channels]/1E3, themisd_spectra_med
                markersize=7, elinewidth=1, marker='.', ls='--')
bx_col.set(yscale='log', xscale='log')
bx_col.text(
    0.99, 0.99,
    f'TH {themisa_spectra_time_ranges[j][0]:%H:%M:%S} - {themisa_spectra_ti
    va='top', ha='right', transform=bx_col.transAxes, color=themis_color)
bx_col.text(
    0.99, 0.9,
    f'EL {elfin_spectra_time_ranges[i][0]:%H:%M:%S} - {elfin_spectra_time_r
    va='top', ha='right', transform=bx_col.transAxes, color=elfin_color)
for bx_col in bx[-1, :]:
    bx_col.set_xlabel('Energy [keV]')
for bx_col in bx[:, 0]:
    bx_col.set_ylabel(f'Electron flux\n{pad_obj._flux_units}')
bx[0,0].set_ylim(1E0, 1E9);

```



Takeaways:

1. THEMIS-A observed an electron spectrum that was consistent with ELFIN-A's RB or IB spectrum (there the RB/IB spectrum difference is negligible).
2. THEMIS-D observed an electron spectrum that was consistent with ELFIN-A's PS spectrum (although the spectrum deviates above 200 keV---is 10^3 - 10^4 the SST noise floor for > 200 keV channels?)